

CSCI 347 Project 02: Exploring Graph Data

You are required to do this in groups of 3 students — preferably using the groups that were formed for project 1. If you want to form a new group let me know. For this Mini project we are going to use datasets from the following data repositories.

- The **SNAP collection** at Stanford offers various datasets for large-scale network analysis, including social, communication, and citation networks, along with temporal datasets. These datasets are often used for studying graph properties, community detection, and network dynamics.
 - <https://snap.stanford.edu/data>
- The **Network Repository** provides a collection of real-world and synthetic network datasets for research in graph mining, machine learning, and complex systems.
 - <https://networkrepository.com/index.php>

Pick a dataset from one of these data repositories.

Run all the analysis on the largest connected component of the graph.
--

Note that these datasets could be quite large. If analyzing data takes too long, then you may preprocess the dataset by taking a sample of the graph and then extracting the largest connected component so that you can get the graph to a manageable size.

Tips

If you like, you can use a Jupyter notebook file and write the answers in the cells (using markdown) and include your code. Also, you can submit the code and PDF report separately. If you do so, use screenshots to show the functions and your answers.

Problem 1

First, you need to be familiar with the dataset. Answer the following questions in well-written paragraphs.

1. [4 points] Why did you **choose** this dataset, and why is it **interesting** to you? (Make sure you include a **link** to the dataset that you chose).
2. [3 points] Did you **preprocess** the data? If so, how did you preprocess it? Clearly explain the process. e.g., Did you take the largest connected component? Was it manageable, if not, did you sample it? How did you take the sample of the graph? What did you do after sampling? Did you do any other preprocessing? etc.
3. [2 points] Explain what type of data is represented in this dataset.
4. [6 points] Answer the following questions before doing the analysis.
 - a. [2 points] What characteristic do you expect the nodes with high centrality to possess? E.g., what type of non-graph characteristic would you expect the nodes with high centrality have? Note that this answer depends on the dataset that you choose.
 - b. [2 points] Do you think this graph dataset would exhibit power law? State your reasons.

- c. [2 points] Do you expect this graph to have small-world property? State your reasons.

Problem 2

Implement the following functions in **Python**. You can assume that the graph is **simple**, i.e., the graph does not have self-loops and parallel edges, undirected, unweighted. You can use networkx libraries as long as the function does not perform the same task as what you are being asked to implement. For example, if you are asked to implement a function that calculates betweenness centrality, then you can use networkx function to find shortest paths, but you cannot use the networkx function that calculates the betweenness centrality in your implementation. Also, you can assume that nodes labels are **integers**. **Test each of your functions on facebook_combined dataset.**

1. [5 points] Implement a function that takes list of edges of a graph—represented as tuples—and output the **number of nodes**.
2. [5 points] Implement a function that takes list of edges of a graph—represented as tuples—and an input vertex index as input and output the **degree of the input vertex**.
3. [5 points] Implement a function that takes list of edges of a graph—represented as tuples—and output the **adjacency matrix** of the graph.
4. [5 points] Implement a function that takes list of edges of a graph—represented as tuples—and output the **degree distribution of the graph**. Output the degree distribution as a python list.
5. [5 points] Implement a function that takes **degree distribution of a graph**—represented as a list—and **input degree**, then output the **probability of picking a vertex with degree higher than or equal** to the input degree.
6. [5 points] Implement a function that takes input vertex index, and a list of edges of a graph—represented as tuples—and output the **eccentricity** of the given vertex.
7. [5 points] Implement a function that takes list of edges of a graph—represented as tuples—and output the **diameter of the graph**. (you can use networkx function to find the shortest path between two vertices)
8. [5 points] Implement a function that takes list of edges of a graph—represented as tuples—and output the **radius of the graph**. (you can use networkx function to find the shortest path between two vertices)
9. [5 points] Implement a function that takes vertex index and a list of edges of a graph—represented as tuples—and output the **clustering coefficient** of the input vertex.
10. [5 points] Implement a function that takes a vertex index and a list of edges of a graph—represented as tuples—and output the **betweenness centrality** of the input vertex.
11. [5 points] Implement a function that takes a vertex index and a list of edges of a graph—represented as tuples—and output the **closedness centrality** of the input vertex.
12. [5 points] Implement power iteration to find the **eigenvector centrality** of each node in a network. But there is a **slight difference in the implementation**. Instead of using L_∞ norm for normalizing the resultant vector at each iteration use L_2 (Euclidean distance) for normalization. You **may** use numpy matrix multiplication library functions, transpose function, dot product functions, norm functions but you **may not** use the libraries to directly calculate eigenvectors. You may assume that the inputs to this function are the adjacency matrix and max number of iterations.

Problem 3

Report the following, using tables or figures as appropriate. You may treat the graph as an **undirected, unweighted** graph with no loops or parallel edges. **You may use any networkx functions for all of Part 3**, but you are encouraged to use your functions from Part 2 on real-world data.

1. [5 points] Provide a visualization of the graph dataset that you chose. (If the dataset is too large, you may visualize a sample of the graph.)
2. [3 points] List the 10 nodes with highest closedness centrality.
3. [3 points] List the 10 nodes with highest betweenness centrality.
4. [3 points] List the 10 nodes with highest clustering coefficients.
5. [3 points] Find the top 10 nodes ranked by the highest prestige/eigenvector centrality.
6. [3 points] List the top 10 nodes ranked by PageRank.
7. [4 points] Go through your answers on Problem 3 [2-6] sub questions. Are these highly ranked nodes based on different centrality measurements the same or different? If there is a difference, is it significant or not? Why do you think this is the case?
8. [3 points] Plot the log-log plot of the degree distribution of the graph—make sure that x-axis is the $\log k$ and y-axis is the $\log f(k)$, where $f(k)$ is the probability that a random node we pick has degree k . Make sure you draw **the least-square fit** line of the given data points as well. By looking at the plot, do you think the graph exhibits power-law behavior?

Acknowledgements

Project adapted from assignments of Dave Millman and Veronika Strnadova-Neely.